

Government agencies, index-linked bonds, and what a linker's number means

Date: 2026-09-10 · **Valuation date:** 31 March 2009 (10 June 2009 kept as a control)

Who should read what. Sections 1 to 5 are written for everyone and assume no bond-market vocabulary — every technical term is explained in the sentence where it first appears. **Section 6 is the engineering section:** what moved, how it is proved, and how to run it. A finance reader can skip section 6 entirely and miss nothing.

One thing to say up front. This week's work deliberately changed **no number at all**. Every price, spread and duration this system publishes is byte-for-byte what it was before. That is the point of the exercise and it is proved rather than asserted (section 5) — but it means the interesting part of this report is not the restructuring. It is **section 4**, which explains what one of these classes has been telling us since July and has never appeared in a report.

1. What you asked for

Two classes, brought onto the code structure you approved in August — the one built for the Google team to take over: many small functions, every input named and numbered, the mathematics separated from the products that use it.

- **Government Agencies** — bonds issued by institutions that are not the government itself but are closely tied to it: the Federal Home Loan Banks, the Korea Development Bank, Pemex. **42 lines in the holdings file → 39 distinct securities.**
- **Index Linked Government Bonds** — government bonds whose payments rise with inflation, so the holder is protected against prices going up. Also called linkers, or TIPS in the United States. **16 lines → 15 distinct securities.**

We added a third class, and the reason is worth a paragraph

Guaranteed Fixed Income — 11 lines, **9 securities** — went in with them.

These three classes are not three separate things in our code. They share one piece of software that reads the holdings file, one program that prices them, and **one output file containing all 63 of them.**

Restructuring two of the three would have left the third stranded: old-style code inside new-style code, with two files each owning half of a decision. That specific shape has caused five separate defects in this project, every one of them silent, so we do not create new ones.

Guaranteed Fixed Income is nine ordinary bonds and one reporting rule. It cost almost nothing to bring along, and leaving it out would have cost a seam.

69 lines, 63 securities, before and after.

2. The three classes needed very different amounts of work, and we should be clear about that

It would be easy to write "we restructured two classes" and let that stand. It would also be misleading, so here is the honest split.

class	what actually happened
Index-Linked	The only real migration. The pricing engine moved into the shared structure and gained a proper front end.
Government Agencies	No engine moved, because it never had one of its own.
Guaranteed	One reporting rule, written down and tested.

Why agencies needed no engine

The 39 agency securities are priced five different ways, and **every one of those five was already built and already migrated** in the two rounds before this one:

how it is priced	how many	
ordinary fixed-coupon bond	27	migrated in the August sample
option tree (the issuer may repay early)	5	migrated in round 2a
its early-repayment date has already passed, so it is an ordinary bond	4	same engine as the first row
a bond that pays no interest, only a lump sum at the end	2	same engine, with the coupon set to zero
a mortgage security filed in the wrong category	1	not priced — see below

So what changed for agencies is the **wrapper and the labelling**, not the mathematics. That is a smaller claim than "we restructured a class", and it is the true one.

The one agency security we do not price, and why we leave it alone

One holding, described as `FHLMC SER 3122 CL ZB`, is filed under Government Agencies but is not an agency bond at all: it is a slice of a pool of mortgages, of a kind whose interest is added to the principal rather than paid out. Pricing it with a bond model would produce a number that looks perfectly reasonable and means nothing.

It is carried at the custodian's own price and will be priced properly in the mortgage phase. This is the same rule we applied to the Sempra bond in July: **when the terms do not fit the model, we say so rather than force a number.**

3. Where this leaves the book

Unchanged from last week, because this week moved no securities:

category	lines	securities	status
Corporate Bonds	811	732	done — 566 priced
Government Bonds	153	147	done
Government Agencies	42	39	done · restructured this week
Index Linked Government Bonds	16	15	done · engine migrated this week
Guaranteed Fixed Income	11	9	done · restructured this week
Municipal/Provincial Bonds	7	7	done
all six ordinary cash-bond classes	1,040	949	complete
Government MBS	888	882	engine built, waiting on the Bloomberg data
Non-Government CMOs	265	264	not started
Asset Backed Securities	79	79	not started
Commercial Mortgage-Backed	73	69	not started
derivatives and other	21	17	out of scope
total	2,366	2,260	

4. What an inflation-linked bond has been telling us since July

This is the part of the report we would most like you to read, and it needs no code in it.

4.1 The number these bonds produce is not a credit spread

For an ordinary bond, our system works out the extra yield a buyer demands over and above government interest rates. That extra yield is compensation for the risk the borrower does not pay — a **credit spread**.

For an inflation-linked bond the same arithmetic produces something completely different, and the reason is simple. These bonds pay more when prices rise, and we price them on ordinary interest rates **with an explicit assumption about future inflation — set to zero**. So the model is deliberately projecting cash flows that leave inflation out. When we then match that model to the real market price, the leftover has to absorb exactly the inflation the projection omitted.

The consequence: **the number comes out negative, and it is approximately minus the inflation rate the market expects**. A negative number here is the engine working, not an error.

Turned the right way up, it is the **breakeven inflation rate**: the rate of inflation at which an investor would be indifferent between an inflation-linked bond and an ordinary one. That is a genuine market observation, extracted per bond, for free.

Because this number is so easily mistaken for a credit spread, it lives in **its own column** with its own name and can never be averaged in with credit spreads. We have gone further this week: the software will now **refuse to build** if anyone renames the function to something credit-shaped. (Engineering detail in section 6.)

4.2 At the end of March 2009, these bonds were pricing deflation

Read off the run, across thirteen holdings:

bond matures	breakeven inflation the market was pricing
January 2010	–34 bp — that is, falling prices
July 2014	+35 bp
January 2017	+69 bp
January 2025	+85 bp
April 2032	+139 bp

The shape is the crisis itself. In March 2009 the market expected prices to **fall** over the coming year, and expected inflation to return only slowly over decades. A central bank looking at that curve sees a deflation scare written down in prices.

The median across the thirteen was 85 basis points — that is, 0.85% a year.

4.3 Ten weeks later the scare had largely unwound

Re-running everything at 10 June 2009, our control date:

	31 March 2009	10 June 2009
median breakeven inflation	85 bp	215 bp
the January 2010 bond	–34 bp	+60 bp

The near-dated bond is the striking one: in March the market was paying for protection against **falling** prices within the year; by June it had stopped. Between those two dates the recovery that resolved your question about the custodian's pricing in July is visible in a completely independent place — the inflation market rather than the credit market. Two different measurements, the same story.

4.4 The Japanese bond goes the other way, and that is the check that it works

One holding is a Japanese inflation-linked bond. Its breakeven comes out at **–229 basis points** — about **–2.3% a year**.

Japan in 2009 had been in deflation for a decade, and the market expected that to continue. So the sign flips, and it flips in the direction the economics demand. A model that produced the same sign everywhere would be telling us about itself rather than about the market; this one does not.

5. How we know nothing broke

The claim "we changed the structure and not the numbers" is easy to make and easy to get wrong, so it is proved mechanically.

Before touching anything, we took a cryptographic fingerprint of every file the system publishes — thirteen of them — and confirmed those fingerprints matched the ones recorded in the previous release. The files on disk were the record, not a later copy of it.

After the work, all four pricing programs were re-run from scratch and every file fingerprinted again. **All twenty-two files in the output folder are identical to the byte.**

Alongside that, the automatic checks went from **424 to 468**. The additions are not padding:

- 32 check the new structure, including one that *fails on purpose* if anyone gives the inflation-linked function a credit-sounding name — we verified it fails by making it fail;
- 3 check that the code map at the front of the package is telling the truth (section 6.3);
- 2 check that the new safety guard runs **before** any calculation rather than after;
- 7 wrote themselves — one existing check runs once per source file, so it picked up the new modules without anyone asking.

6. Engineering section

A finance reader can stop here.

6.1 What moved

`src/pricing/ilb.py` → `src/pricer/core/pricing/inflation.py`. The body below the docstring was **spliced, not retyped**, and hashes identically to the source (105 lines, sha256 `ea475752...`). The old path is now a shim, and shim and core are asserted to export the **same objects** — not merely to be importable. A shim that silently re-implements passes every import-by-name test while production drifts onto a second copy; object identity is the only assertion that catches that, and it is how the FRN and lattice shims are pinned too.

New package `src/pricer/assets/government/` :

module	what it owns
<code>bonds_input.py</code>	16 numbered inputs, its own numbering — government input 9 is the linker spread, corporate input 9 is a scenario size. Two dictionaries, neither renumbering the other.
<code>linker.py</code>	seven per-metric functions over the inflation engine, in legacy units
<code>agency.py</code>	the class conventions, and the rule for reading a callable result
<code>guaranteed.py</code>	the FDIC guarantee reporting rule
<code>sovereign.py</code>	the government-side path to the shared option engine

6.2 Two decisions taken before writing, and what they rule out

No endpoint type for inflation-linked bonds this round. The spreadsheet has no cells for a real coupon, an index ratio or an inflation assumption. The project's standing rule is that an instrument type becomes reachable from a worksheet only once you have chosen the layout for its inputs — so the engine and its wrapper exist, the contract is untouched (`schema_version` stays 1.1), and every government input's external field reads `-`. A test pins that to the seven-type contract, so it cannot drift open by accident. **It is ready to connect the day the sheet has the cells.**

The shared option engine was not moved. One binomial tree prices every bond with an embedded option in this book — corporate callables, agency debentures and one callable US Treasury. It currently lives under `assets/corporate/`, which is a naming problem when a government program imports it, so government code now has **its own path to the same objects**. Relocating the module itself would have touched the corporate outputs for a naming gain; it is recorded as a follow-up with a trigger rather than done opportunistically.

6.3 Three things this round found in our own work

The code map was a round out of date. `pricer/__init__.py` is the first file anyone reads on the walkthrough. It still described the option tree, the callable wrapper and the floating-rate wrapper as *planned* — months after they shipped — and called the JSON interface a future idea when it has been live since August. A docstring cannot fail a test, so nothing caught it.

It is rewritten, and three tests now hold it honest. Two are the obvious directions and **neither would have caught this**. The one that matters is: *nothing marked planned may already exist* — the entry was present, parsed correctly, and was simply untrue. Reverting to the historical map turns all three red, which is how we know they bind.

Three unnamed numbers in the pricing program. How a callable agency result gets described depended on three bare constants written inline: a negative spread means our assumed call terms conflict with the price; a gap wider than 100 basis points means the market is pricing the bond to maturity rather than to the call; a gap under 1 basis point means the call never binds. They are now named constants behind one function. The program keeps its own copy so its output stays byte-identical, and the price of that choice is that the rule lives in two places — so a test reads the program's source and fails if they ever diverge. (*At 31 March all five callable agencies read "call-active".*)

A safety guard that found nothing, reported as finding nothing. A call date falling inside a bond's final coupon period lands on no point of the model's grid; the bond then prices silently as if it had no call at all, and reports an option value of exactly zero — because the option was never evaluated. We found that on the corporate side in August. The agency program was the last place without the check.

It now has it. **All five agency schedules reach a valid point, at both dates; nothing was refused; both files are byte-identical.** The guard is **latent, not live**. We report that as what it is rather than as a fix, and had it refused a bond, the bond would have appeared in the output with a named reason — never as a missing row.

6.4 Running it

```
python -m pytest -q
```

468 checks, ~49 s

```
python scripts/phase2_risk.py
```

the three classes, 63 securities

`FIP_VAL_DATE` selects the valuation date, `FIP_OUT` the output file, `FIP_VOL` the volatility for the option tree (0.15), `FIP_INFL` the inflation assumption (0 — which is what makes the calibrated spread readable directly as a breakeven rate).

7. Open questions for you

1. **Should inflation-linked bonds become available from the spreadsheet?** If yes, we need to know where you want three inputs to live on the sheet: the real coupon, the index ratio and the inflation assumption. The engine is ready; only the layout is your decision.
2. **Nothing else is blocked.** The mortgage data request from July is still the one thing holding up the remaining 1,294 securities, and this week did not add to it.